DESENVOLVIMENTO MOBILE · MÓDULO 12 · TRILHA KOTLIN · NÍVEL INTERMEDIÁRIO

Firebase: Backend como Serviço

Authentication, Cloud Firestore, regras de segurança, Storage, Cloud Messaging, Crashlytics e alternativas

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar o modelo *Backend as a Service* e os principais serviços do Firebase.
- Configurar um projeto Firebase em um app Android.
- Autenticar usuários com e-mail/senha e conta Google.
- Modelar, gravar e observar dados em tempo real no Cloud Firestore.
- Proteger os dados com regras de segurança e testá-las no emulador local.
- Enviar arquivos ao Storage, receber notificações push e monitorar falhas.

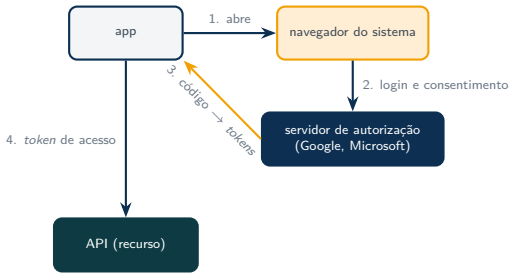
Roteiro

Neste módulo

- Fundamentos: identidade, tempo real e mensagens
- Visão geral
- Configuração
- Authentication
- Cloud Firestore
- Regras de segurança
- Storage, mensagens e monitoramento
- Fechamento

Fundamentos: identidade, tempo real e mensagens

Identidade federada: OAuth 2.0 e OpenID Connect



Um pouco de história

OAuth 2.0 virou padrão em 2012 (RFC 6749); o OpenID Connect veio em 2014. Para apps nativos, a RFC 8252 (2017) recomenda o navegador do sistema e PKCE.

OAuth 2.0 e OpenID Connect

OAuth 2.0 delega **autorização**: o app recebe um *token* para agir em nome do usuário sem ver a senha. O **OpenID Connect** acrescenta **identidade**: um *ID token* diz quem é o usuário.

PKCE

O app cria um segredo aleatório por login e envia só o seu *hash*; na troca do código pelos *tokens*, mostra o segredo. Quem interceptar o código não consegue usá-lo.

Atenção

Nunca peça a senha do Google numa tela do app nem use *WebView* para o login.

Tokens no app: identidade, acesso e renovação

Token	Para que serve	Validade típica	Cuidados
ID *token*	informa quem é o usuário (nome, e-mail, `sub`)	1 hora no Firebase	não serve para chamar APIs de terceiros
De acesso	autoriza chamadas a uma API	minutos a 1 hora	enviar só por HTTPS, no cabeçalho `Authorization`
De renovação	obtém novos *tokens* sem novo login	semanas a meses	guardar em armazenamento seguro; revogar no *logout*

Vida curta, renovação automática

Tokens de acesso curtos limitam o estrago se vazarem. Os SDKs do Firebase renovam o ID *token* sozinhos antes de expirar.

Papéis dentro do *token*

O servidor pode gravar papéis no *token* (*custom claims*, como `professor: true`). As regras de segurança e as funções leem esses papéis sem consultar o banco.

Senhas, múltiplos fatores e *passkeys*

Fatores de autenticação

Algo que você **sabe** (senha), algo que você **tem** (celular, chave física) e algo que você **é** (digital, rosto). Combinar dois fatores diferentes é a autenticação multifator (MFA).

O ponto fraco da senha

Senhas são reutilizadas, vazam e podem ser digitadas num site falso (*phishing*). Códigos por SMS ajudam, mas também podem ser interceptados.

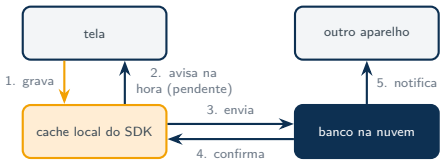
Passkey

Credencial baseada em par de chaves, criada para um único site ou app. A chave privada fica no aparelho ou no gerenciador de senhas; o servidor guarda só a pública.

Por que resiste a *phishing*

O login é a assinatura de um desafio do servidor, liberada pela biometria ou pelo PIN do aparelho. A chave só funciona no domínio para o qual foi criada, e não há segredo para vazar do servidor.

Bancos em tempo real: escuta, cache local e compensação de latência



Escuta (*listener*)

Em vez de perguntar repetidas vezes, o app assina uma consulta e recebe o resultado inicial e depois cada mudança, por uma conexão mantida aberta.

Compensação de latência

A gravação aparece na tela antes de o servidor confirmar, marcada como pendente. Se o servidor recusar, o SDK desfaz a mudança.

Escutas custam

Cada documento recebido conta como leitura. Limite as consultas (filtros, `limit`) e encerre as escutas quando a tela sair.

Regras de segurança: a autorização mora no servidor

O app fala direto com o banco

Em um BaaS não há servidor seu no meio: qualquer pessoa pode usar as chaves públicas do app em outro programa. Verificações feitas no app são só experiência de uso; a proteção real são as **regras**, avaliadas no servidor a cada requisição.

firestore.rules (trecho)

```
match /reservas/{id} {
  allow read: if request.auth.uid == resource.data.uid;
  allow create: if request.auth.uid
                   == request.resource.data.uid;
}
```

Menor privilégio

Comece negando tudo e libere só o necessário, para quem precisa. Valide também o formato: campos obrigatórios, tipos e tamanhos.

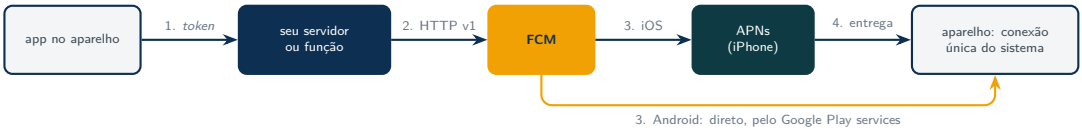
Regras não são filtros

A regra não esconde documentos de uma consulta: se a consulta **puder** trazer algo proibido, ela inteira falha. A consulta precisa ter os mesmos filtros da regra.

Dica

Teste as regras no emulador, com casos permitidos e negados, como qualquer outro código.

Notificações *push* por dentro



Token de registro

Endereço do app naquele aparelho. Muda em reinstalações e após limpar dados: mantenha-o atualizado no servidor.

Dois tipos de mensagem

Notificação: o sistema exibe sozinho. **Dados**: o código do app decide o que fazer.

Entrega não garantida

O usuário pode negar a permissão (Android 13+ e iOS), e o sistema adia mensagens para poupar bateria.

Funções acionadas por eventos



Dica

Outros gatilhos: requisição HTTP, conta criada, arquivo enviado e horário agendado.

Arquitetura orientada a eventos

Componentes reagem a fatos (“uma leitura foi gravada”) em vez de chamarem uns aos outros. Quem grava não precisa saber quem reage.

Pelo menos uma vez

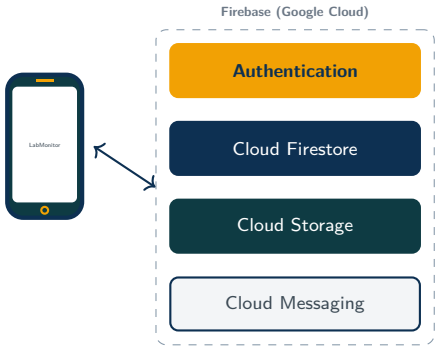
O provedor garante executar a função ao menos uma vez por evento, e às vezes executa duas. Escreva funções **idempotentes**: processar o mesmo evento de novo não pode duplicar o *push* nem o resumo.

Onde guardar segredos

Chaves de serviços pagos ficam no servidor e nunca no app. O App Check (Play Integrity, App Attest) ajuda a recusar chamadas que não vêm do app legítimo.

Visão geral

Backend as a Service



O **Firebase** oferece, como serviços gerenciados, as peças que todo app conectado precisa:

- login de usuários;
- banco de dados em tempo real com cache offline;
- armazenamento de arquivos;
- notificações push;
- relatórios de falhas e métricas de uso.

Dica

Ótimo para protótipos, TCCs e apps pequenos e médios: você se concentra no app e não em servidores.

Serviços e planos

Serviço	Para quê
Authentication	login (e-mail, Google, telefone, anônimo...)
Cloud Firestore	banco NoSQL de documentos, tempo real
Cloud Storage	fotos, PDFs, arquivos grandes
Cloud Messaging (FCM)	notificações push
Cloud Functions	código no servidor disparado por eventos
Crashlytics	relatórios de falhas em produção
Analytics	eventos e funis de uso
Remote Config	mudar parâmetros sem publicar versão
App Check	garante que só o seu app acessa o backend
Firebase AI Logic	acesso a modelos Gemini a partir do app (Módulo 20)

Planos

Spark: gratuito, sem cartão; ao atingir a cota, o serviço para até o próximo período.
Blaze: pague pelo uso, com as mesmas cotas gratuitas.

Cloud Storage exige Blaze

Desde **03/02/2026**, o Cloud Storage para Firebase só funciona em projetos no plano Blaze (a cota gratuita continua valendo). Ao ativar o Blaze, configure um **alerta de orçamento**.

Configuração

Adicionando o Firebase ao app

1. Crie um projeto no console do Firebase.
2. Adicione um app Android com o `applicationId` do app.
3. Informe o SHA-1 de debug (`./gradlew signingReport`), exigido pelo login Google.
4. Baixe o `google-services.json` para `app/`.
5. Adicione o plugin e as bibliotecas.

Dica

Tools → **Firebase** no Android Studio automatiza esses passos.

app/build.gradle.kts

```
plugins {
  alias(libs.plugins.google.services)
  alias(libs.plugins.crashlytics)
}
dependencies {
  // o BoM alinha as versões de tudo
  implementation(platform(libs.firebase.bom))
  implementation(libs.firebase.auth)
  implementation(libs.firebase.firestore)
  implementation(libs.firebase.storage)
  implementation(libs.firebase.messaging)
  implementation(libs.firebase.crashlytics)
}
```

Atenção

Os antigos módulos `-ktx` foram descontinuados: as APIs Kotlin (`Firebase.auth`, `snapshots()`...) agora estão nos módulos principais.

Authentication

Login com e-mail e senha

AuthRepository.kt

```
class AuthRepository @Inject constructor() {
  private val auth = Firebase.auth
  // usuário atual como Flow (null = deslogado)
  val usuario: Flow<FirebaseUser?> = callbackFlow {
    val l = FirebaseAuth.AuthStateListener {
      trySend(it.currentUser)
    }
    auth.addAuthStateListener(l)
    awaitClose { auth.removeAuthStateListener(l) }
  }
  suspend fun entrar(email: String, senha: String) =
      runCatching {
    auth.signInWithEmailAndPassword(email, senha).await()
  }
  suspend fun cadastrar(email: String, senha: String) =
      runCatching {
    auth.createUserWithEmailAndPassword(email, senha).await()
  }
  fun sair() = auth.signOut()
}
```

await()

As APIs do Firebase devolvem `Task`. A extensão `await()` (de `kotlinx-coroutines-play-services`) as transforma em chamadas `suspend`.

Navegação condicionada

`usuario` nulo leva ao login; senão, ao painel. A sessão persiste entre aberturas.

Atenção

O app nunca guarda a senha.

Login com Google via Credential Manager



Credential Manager

Unifica senhas, *passkeys* e login Google numa única folha de seleção (substitui o antigo *Google Sign-In*).

Atenção

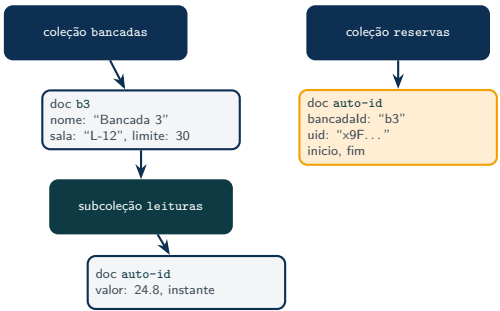
Cadastre o SHA-1 de cada chave que assina o app: debug, upload e Play App Signing (Módulo 15).

Passos

1. Ative o provedor Google no console.
2. Pegue o *Web client ID* do projeto.
3. Peça uma credencial `GetGoogleIdOption` ao `CredentialManager`.
4. Troque o ID token por uma credencial Firebase.

Cloud Firestore

Modelo de dados: coleções e documentos



- **Documento**: conjunto de campos (como um JSON), até 1 MiB.
- **Coleção**: conjunto de documentos.
- Documentos podem ter **subcoleções**.
- Não há *joins*: modele pensando nas **consultas** que a tela fará.

Dica

É comum **duplicar** dados (ex.: o nome da bancada dentro da reserva) para evitar leituras extras. O custo é cobrado por documento lido.

Gravando e lendo em tempo real

ReservaRepository.kt

```
data class ReservaDoc(        // construtor vazio p/ Firestore
  val bancadaId: String = "", val uid: String = "",
  val inicio: Timestamp? = null, val fim: Timestamp? = null)

class ReservaRepository @Inject constructor() {
  private val col = Firebase.firestore.collection("reservas")
  suspend fun criar(r: ReservaDoc): String =
    col.add(r).await().id
  // reemite a cada mudança no servidor ou no cache
  fun daBancada(id: String): Flow<List<ReservaDoc>> = col
    .whereEqualTo("bancadaId", id)
    .whereGreaterThan("fim", Timestamp.now())
    .orderBy("fim")
    .snapshots()
    .map { it.toObjects<ReservaDoc>() }
  suspend fun cancelar(docId: String) {
    col.document(docId).delete().await()
  }
}
```

`snapshots()`

Transforma o *listener* em tempo real do Firestore em `Flow`. Quando outra pessoa cria uma reserva, todas as telas abertas atualizam.

Offline

Cache local ativo: lê sem rede e envia as escritas quando a conexão volta.

Índices

Filtros em campos diferentes exigem índice composto (o Logcat traz o link).

Transações: evitando reservas duplicadas

Reserva atômica

```
suspend fun reservarHorario(b: String, slot: String,
                            uid: String) {
  val db = Firebase.firestore
  val ref = db.collection("bancadas").document(b)
    .collection("slots").document(slot)  // ex.: 2026-11-03T14

  db.runTransaction { tx ->
    val atual = tx.get(ref)
    if (atual.exists()) {
      throw FirebaseFirestoreException("Horário ocupado",
        FirebaseFirestoreException.Code.ABORTED)
    }
    tx.set(ref, mapOf("uid" to uid,
      "criadoEm" to FieldValue.serverTimestamp()))
  }.await()
}
```

Transação

Lê e grava como uma operação única. Se outro cliente alterar o documento no meio, o Firestore repete a transação, e duas pessoas nunca ficam com o mesmo horário.

Modelagem

Usar o horário como **ID do documento** torna o conflito impossível por construção: só pode existir um `slots/2026-11-03T14`.

Atenção

Transações precisam de conexão com o servidor; não funcionam offline.

Regras de segurança

Regras do Firestore

firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /bancadas/{id} {
      allow read: if request.auth != null;
      allow write: if request.auth.token.admin == true;
    }

    match /reservas/{rid} {
      allow read: if request.auth != null;
      allow create: if request.auth != null
        && request.resource.data.uid == request.auth.uid
        && request.resource.data.fim
             > request.resource.data.inicio;
      allow delete: if resource.data.uid == request.auth.uid;
    }
  }
}
```

O erro mais grave

Publicar com `allow read, write: if true;` (“modo de teste”). Qualquer pessoa na internet lê e apaga todo o banco.

Como pensar

O app **não é confiável**: as regras, que rodam no servidor, são a única barreira real.

Emulator Suite

`firebase emulators:start` roda Auth, Firestore e Storage localmente, sem custo e sem dados reais.

Storage, mensagens e monitoramento

Cloud Storage: fotos dos equipamentos

FotoRepository.kt

```kotlin
suspend fun enviarFoto(patrimonio: String, uri: Uri): String {
  val ref = Firebase.storage.reference
    .child("equipamentos/$patrimonio.jpg")
  ref.putFile(uri).await()                    // upload
  return ref.downloadUrl.await().toString()
}

// No ViewModel: grava a URL no documento do equipamento
val url = fotos.enviarFoto("IFMA-0042", uriEscolhida)
equipamentos.atualizarFoto("IFMA-0042", url)
```

Dica

Comprima a imagem antes de enviar (ex.: 1280 px, JPEG 80 %). Fotos de câmera têm vários MB.

storage.rules

```
match /equipamentos/{arquivo} {
  allow read: if request.auth != null;
  allow write: if request.auth != null
    && request.resource.size < 5 * 1024 * 1024
    && request.resource.contentType
         .matches('image/.*');
}
```

Conceito

Arquivos ficam no Storage; o Firestore guarda apenas a URL e os metadados.

Cloud Messaging: notificações push



AndroidManifest.xml

```
<uses-permission android:name=
  "android.permission.POST_NOTIFICATIONS"/>
```

MensagensService.kt

```
class MensagensService : FirebaseMessagingService() {
  override fun onNewToken(token: String) {
    // envie ao seu backend / Firestore
    salvarTokenDoUsuario(token)
  }
  override fun onMessageReceived(msg: RemoteMessage) {
    val titulo = msg.notification?.title ?:
      "LabMonitor"
    mostrarNotificacao(titulo, msg.data["texto"])
  }
}
```

Android 13+

Notificações exigem a permissão em tempo de execução `POST_NOTIFICATIONS` (Módulo 13). Registre o *service* no manifesto.

Crashlytics, Analytics e alternativas ao Firebase

Crashlytics

Agrupa as falhas de produção por causa, com pilha de chamadas, modelo do aparelho e versão do Android. Use `Firebase.crashlytics.log("...")` e `setCustomKey` para dar contexto.

Analytics

`logEvent("reserva_criada")` registra eventos e mede o uso real das funcionalidades.

LGPD

Informe na política de privacidade quais dados são coletados e por quê (Módulo 15).

Alternativa	Características
Supabase	PostgreSQL (SQL e relações), auth, storage, tempo real; código aberto
Appwrite	código aberto, pode ser hospedado no próprio servidor
Backend próprio	Ktor, Spring, FastAPI ou Node com banco relacional; controle total, mais trabalho

Mantenha a porta aberta

Com a arquitetura do Módulo 08, o Firebase fica atrás de interfaces de repositório. Trocar de backend afeta só a camada de dados.

Fechamento

Resumo

Resumo

- Firebase entrega autenticação, banco, arquivos, push e monitoramento sem servidor próprio.
- Configure com `google-services.json`, o plugin e o BoM; use `await()` e `snapshots()` com coroutines.
- Firestore: coleções e documentos, consultas com índices, cache offline e transações.
- As regras de segurança são a proteção real dos dados; teste-as no Emulator Suite.
- Storage guarda arquivos (exige plano Blaze); FCM entrega notificações push.
- Isole o Firebase atrás de repositórios para poder trocar de backend.

Glossário (1/2)

Termo	Significado
BaaS	*backend* pronto oferecido como serviço: contas, banco, arquivos
Firebase Authentication	serviço de contas e login (e-mail, Google, telefone)
OAuth 2.0	delegação de autorização por *tokens*, sem expor a senha
OpenID Connect	camada de identidade sobre o OAuth (ID *token*)
PKCE	segredo por login que impede o uso de um código interceptado
ID *token*	JWT que diz quem é o usuário; vale 1 hora no Firebase
Custom claims	papéis gravados no *token* pelo servidor
Passkey	credencial de par de chaves, resistente a *phishing*
MFA	autenticação com dois ou mais fatores diferentes

Glossário (2/2)

Termo	Significado
Cloud Firestore	banco de documentos com sincronização em tempo real
Listener	assinatura que recebe o resultado e cada mudança de uma consulta
Compensação de latência	a gravação aparece antes da confirmação do servidor
Regras de segurança	autorização declarativa avaliada no servidor a cada requisição
FCM	serviço de mensagens *push* do Firebase
***Token* de registro**	endereço do app em um aparelho, para receber *push*
Idempotência	processar o mesmo evento duas vezes dá o mesmo resultado
Crashlytics	relatório de falhas do app em produção

Questões de revisão

1. Diferencie OAuth 2.0 de OpenID Connect. Qual deles o "Entrar com Google" usa?
2. Por que apps nativos devem fazer login no navegador do sistema, com PKCE?
3. Para que servem o ID *token*, o *token* de acesso e o *token* de renovação?
4. Por que *passkeys* resistem a *phishing* e senhas não?
5. O que é compensação de latência? Como a tela sabe que um documento ainda está pendente?
6. Por que as regras do Firestore são a verdadeira proteção dos dados? O que significa "regras não são filtros"?
7. Descreva o caminho de um alerta *push* da função até um iPhone e até um Android.
8. Por que uma função acionada por evento precisa ser idempotente?

Exercícios

Exercício 1 — Autenticação

Implemente as telas de login, cadastro e "esqueci a senha" (`sendPasswordResetEmail`) com validação e mensagens de erro em português.

Exercício 2 — Regras

Escreva regras que permitam a qualquer usuário logado ler as bancadas, mas só o dono editar ou cancelar a própria reserva. Teste no emulador com dois usuários.

Exercício 3 — Tempo real

Abra o app em dois aparelhos com usuários diferentes. Crie uma reserva em um e observe o outro. Desligue a rede de um deles e explique o comportamento.

LabMonitor — Etapa 12

Adicione login, reservas de bancada no Firestore com transação anti-conflito e regras de segurança. Integre o Crashlytics.

Para aprofundar

- Firebase para Android: `https://firebase.google.com/docs/android/setup`
- Authentication: `https://firebase.google.com/docs/auth/android/start`
- Credential Manager: `https://developer.android.com/identity/sign-in/credential-manager`
- Cloud Firestore: `https://firebase.google.com/docs/firestore`
- Regras de segurança: `https://firebase.google.com/docs/rules`
- Emulator Suite: `https://firebase.google.com/docs/emulator-suite`
- Preços: `https://firebase.google.com/pricing`

Respostas das questões de revisão

1. O OAuth 2.0 delega autorização (*token* de acesso); o OpenID Connect acrescenta identidade (ID *token*). O login com Google usa OpenID Connect: o app recebe um ID *token* e o troca por uma credencial do Firebase.
2. O navegador do sistema mantém a senha longe do app e reaproveita a sessão já aberta; o PKCE impede que um código interceptado seja trocado por *tokens*.
3. ID *token*: quem é o usuário. Acesso: autoriza chamadas a uma API. Renovação: obtém novos *tokens* sem novo login.
4. A *passkey* só funciona no domínio para o qual foi criada e assina um desafio sem enviar segredo. A senha pode ser digitada num site falso e reutilizada.
5. A gravação aparece na hora, antes da confirmação do servidor; se ele recusar, o SDK desfaz a mudança. Enquanto isso, `metadata.hasPendingWrites` é verdadeiro.
6. O app fala direto com o banco e qualquer um pode usar as chaves públicas: só as regras, avaliadas no servidor, barram o acesso. Uma consulta que **possa** trazer documentos proibidos falha inteira; ela precisa ter os mesmos filtros da regra.
7. A função chama o FCM pela API HTTP v1 com o *token* do aparelho. No Android, o FCM entrega pelo Google Play services; no iPhone, repassa ao APNs, que entrega pela conexão do sistema.
8. O provedor garante pelo menos uma execução por evento e às vezes repete; sem idempotência, o mesmo alerta seria enviado duas vezes.

Fim do Módulo 12

Próximo: Módulo 13 — Recursos do dispositivo e IoT: permissões, câmera, localização, WorkManager, sensores, BLE e MQTT com ESP32.